

SQL OPERATIONS PRACTICAL

Table Used: CUSTOMERS

```
CREATE TABLE CUSTOMERS (  
  ID INT PRIMARY KEY,  
  NAME VARCHAR(20),  
  AGE INT,  
  ADDRESS VARCHAR(50),  
  SALARY DECIMAL(10,2)  
);
```

```
mysql> DESC CUSTOMERS;  
+-----+-----+-----+-----+-----+-----+  
| Field | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| ID    | int           | NO   | PRI | NULL    |       |  
| NAME  | varchar(20)   | YES  |     | NULL    |       |  
| AGE   | int           | YES  |     | NULL    |       |  
| ADDRESS | varchar(50)   | YES  |     | NULL    |       |  
| SALARY | decimal(10,2) | YES  |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.00 sec)
```

INSERT INTO CUSTOMERS (ID, NAME, AGE, ADDRESS, SALARY) VALUES

(1, 'Prajwal', 21, 'Chandanapuri', 1000000),

(2, 'Prafull', 22, 'Chalisgaon', 90000),

(3, 'Sarathak', 20, 'Niphad', 400000),

(4, 'Shreyas', 21, 'Sangamner', 500000),

(5, 'Avej', 23, 'Loni', 600000);

```
mysql> SELECT * FROM CUSTOMERS;  
+-----+-----+-----+-----+-----+  
| ID | NAME      | AGE | ADDRESS      | SALARY      |  
+-----+-----+-----+-----+-----+  
| 1  | Prajwal  | 21  | Chandanapuri | 1000000.00  |  
| 2  | Prafull  | 22  | Chalisgaon   | 90000.00    |  
| 3  | Sarthak  | 20  | Niphad       | 400000.00   |  
| 4  | Shreyas  | 21  | Sangamner    | 500000.00   |  
| 5  | Avej     | 23  | Loni         | 600000.00   |  
+-----+-----+-----+-----+-----+  
5 rows in set (0.00 sec)
```

a) ALTER TABLE

Used to modify structure of existing table.

-- Add new column

```
ALTER TABLE CUSTOMERS ADD PHONE VARCHAR(15);
```

Field	Type	Null	Key	Default	Extra
ID	int	NO	PRI	NULL	
NAME	varchar(30)	YES		NULL	
AGE	int	YES		NULL	
ADDRESS	varchar(50)	YES		NULL	
SALARY	decimal(10,2)	YES		NULL	
PHONE	varchar(15)	YES		NULL	

6 rows in set (0.00 sec)

-- Modify column

```
ALTER TABLE CUSTOMERS MODIFY NAME VARCHAR(30);
```

Field	Type	Null	Key	Default	Extra
ID	int	NO	PRI	NULL	
NAME	varchar(30)	YES		NULL	
AGE	int	YES		NULL	
ADDRESS	varchar(50)	YES		NULL	
SALARY	decimal(10,2)	YES		NULL	
PHONE	varchar(15)	YES		NULL	

6 rows in set (0.00 sec)

-- Drop column

```
ALTER TABLE CUSTOMERS DROP COLUMN PHONE;
```

```
mysql> DESC CUSTOMERS;
```

Field	Type	Null	Key	Default	Extra
ID	int	NO	PRI	NULL	
NAME	varchar(30)	YES		NULL	
AGE	int	YES		NULL	
ADDRESS	varchar(50)	YES		NULL	
SALARY	decimal(10,2)	YES		NULL	

5 rows in set (0.00 sec)

b) DROP TABLE

DROP TABLE CUSTOMERS;

Deletes entire table permanently.

c) INDEX OPERATIONS

Used to improve query performance.

-- Create index

CREATE INDEX idx_name ON CUSTOMERS(NAME);

SHOW INDEX FROM CUSTOMERS;

```
mysql> SHOW INDEX FROM CUSTOMERS;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
customers	0	PRIMARY	1	ID	A	5	NULL	NULL		BTREE			YES	NULL
customers	1	idx_name	1	NAME	A	5	NULL	NULL	YES	BTREE			YES	NULL

2 rows in set (0.02 sec)

-- Drop index

DROP INDEX idx_name ON CUSTOMERS;

```
mysql> DROP INDEX idx_name ON CUSTOMERS;
ERROR 1091 (42000): Can't DROP 'idx_name'; check that column/key exists
mysql> |
```

d) RELATIONAL OPERATORS (AND, OR)

-- AND Operator

SELECT ID, NAME, SALARY

FROM CUSTOMERS

WHERE SALARY > 2000 AND AGE < 25;

```
WHERE SALARY > 2000 AND
```

ID	NAME	SALARY
1	Prajwal	1000000.00
2	Prafull	90000.00
3	Sarthak	400000.00
4	Shreyas	500000.00
5	Avej	600000.00

5 rows in set (0.00 sec)

```
-- OR Operator
SELECT ID, NAME, SALARY
FROM CUSTOMERS
WHERE SALARY > 2000 OR AGE < 25;
```

ID	NAME	SALARY
1	Prajwal	1000000.00
2	Prafull	90000.00
3	Sarthak	400000.00
4	Shreyas	500000.00
5	Avej	600000.00

5 rows in set (0.00 sec)

e) PATTERN MATCHING (LIKE)

```
-- Salary starting with 200
SELECT * FROM CUSTOMERS
WHERE SALARY LIKE '400%';
```

ID	NAME	AGE	ADDRESS	SALARY
3	Sarthak	20	Niphad	400000.00

1 row in set (0.00 sec)

```
-- Name starts with 'K'
SELECT * FROM CUSTOMERS
WHERE NAME LIKE 'S%';
```

ID	NAME	AGE	ADDRESS	SALARY
3	Sarthak	20	Niphad	400000.00
4	Shreyas	21	Sangamner	500000.00

2 rows in set (0.00 sec)

```
-- Name contains 'a'
SELECT * FROM CUSTOMERS
WHERE NAME LIKE '%a%';
```

```
-- WHERE NAME LIKE '%a%';
```

ID	NAME	AGE	ADDRESS	SALARY
1	Prajwal	21	Chandanapuri	1000000.00
2	Prafull	22	Chalisgaon	90000.00
3	Sarthak	20	Niphad	400000.00
4	Shreyas	21	Sangamner	500000.00
5	Avej	23	Loni	600000.00

5 rows in set (0.00 sec)

f) AGGREGATE FUNCTIONS WITH GROUP BY & HAVING

```
-- Total salary by age
SELECT AGE, SUM(SALARY) AS TOTAL
FROM CUSTOMERS
GROUP BY AGE;
```

```
-- GROUP BY AGE;
```

AGE	TOTAL
21	1500000.00
22	90000.00
20	400000.00
23	600000.00

4 rows in set (0.01 sec)

```
-- Using HAVING
SELECT AGE, SUM(SALARY)
FROM CUSTOMERS
GROUP BY AGE
HAVING SUM(SALARY) > 500000;
```

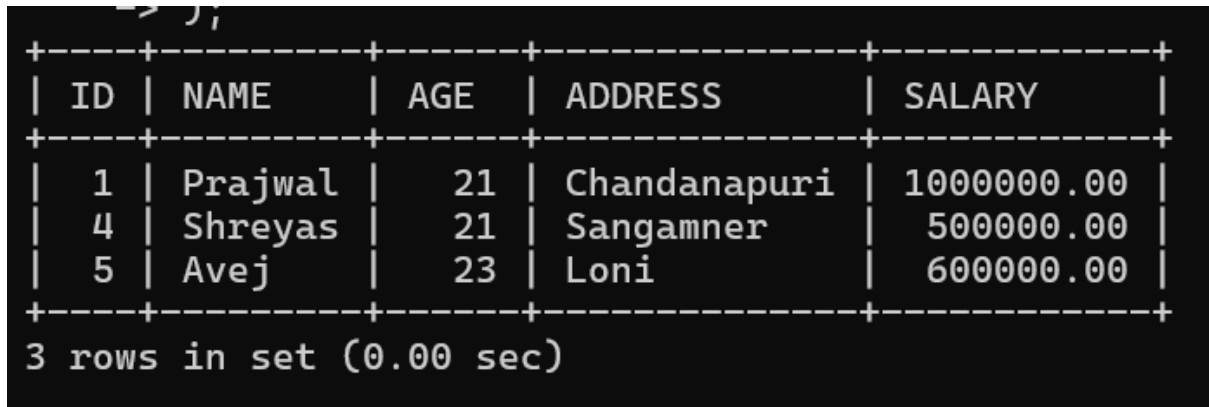
```
-- Using HAVING
```

AGE	SUM(SALARY)
21	1500000.00
23	600000.00

2 rows in set (0.00 sec)

g) NESTED QUERIES

```
SELECT *  
FROM CUSTOMERS  
WHERE ID IN (  
    SELECT ID FROM CUSTOMERS WHERE SALARY > 450000  
);
```



The screenshot shows the output of a SQL query. It displays a table with 6 columns: ID, NAME, AGE, ADDRESS, and SALARY. The table contains 3 rows of data. Below the table, it indicates '3 rows in set (0.00 sec)'.

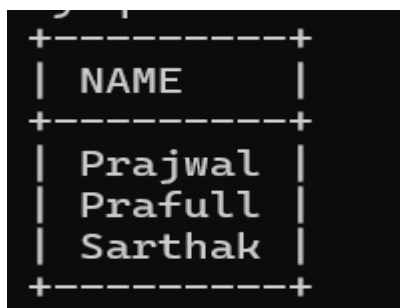
ID	NAME	AGE	ADDRESS	SALARY
1	Prajwal	21	Chandanapuri	1000000.00
4	Shreyas	21	Sangamner	500000.00
5	Avej	23	Loni	600000.00

3 rows in set (0.00 sec)

h) SET OPERATORS

```
CREATE TABLE TABLE1 (  
    NAME VARCHAR(20)  
);
```

```
SELECT * FROM TABLE1;
```



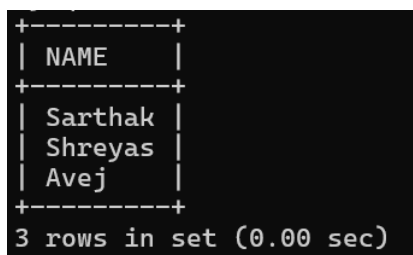
The screenshot shows the output of a SQL query. It displays a table with 1 column: NAME. The table contains 3 rows of data: Prajwal, Prafull, and Sarthak.

NAME
Prajwal
Prafull
Sarthak

```
CREATE TABLE TABLE2 (  
    NAME VARCHAR(20)  
);
```

```
);
```

```
SELECT * FROM TABLE2;
```



The screenshot shows the output of a SQL query. It displays a table with 1 column: NAME. The table contains 3 rows of data: Sarthak, Shreyas, and Avej. Below the table, it indicates '3 rows in set (0.00 sec)'.

NAME
Sarthak
Shreyas
Avej

3 rows in set (0.00 sec)

```
-- UNION
SELECT NAME FROM TABLE1
UNION
SELECT NAME FROM TABLE2;
```

```
+-----+
| NAME |
+-----+
| Prajwal |
| Prafull |
| Sarthak |
| Shreyas |
| Avej |
+-----+
5 rows in set (0.00 sec)
```

```
-- INTERSECT
SELECT NAME FROM TABLE1
INTERSECT
SELECT NAME FROM TABLE2;
```

```
+-----+
| NAME |
+-----+
| Sarthak |
+-----+
1 row in set (0.00 sec)
```

```
-- MINUS
SELECT NAME FROM TABLE1
WHERE NAME NOT IN (SELECT NAME FROM TABLE2);
```

```
+-----+
| NAME |
+-----+
| Prajwal |
| Prafull |
+-----+
2 rows in set (0.00 sec)
```

i) VIEWS

-- Create view

```
CREATE VIEW customer_view AS  
SELECT NAME, SALARY FROM CUSTOMERS;
```

-- Display view

```
SELECT * FROM customer_view;
```

```
mysql> SELECT * FROM customer_view;  
+-----+-----+  
| NAME   | SALARY   |  
+-----+-----+  
| Prajwal | 1000000.00 |  
| Prafull | 90000.00   |  
| Sarthak | 400000.00  |  
| Shreyas | 500000.00  |  
| Avej    | 600000.00  |  
+-----+-----+  
5 rows in set (0.00 sec)
```

-- Drop view

```
DROP VIEW customer_view;
```

```
mysql> DROP VIEW customer_view;  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> SELECT * FROM customer_view;  
ERROR 1146 (42S02): Table 'dbms6.customer_view' doesn't exist  
mysql> |
```

j) SORTING (ORDER BY)

-- Ascending order

```
SELECT * FROM CUSTOMERS  
ORDER BY SALARY ASC;
```

```
+---+-----+-----+-----+-----+  
| ID | NAME   | AGE | ADDRESS       | SALARY   |  
+---+-----+-----+-----+-----+  
| 2  | Prafull | 22  | Chalisgaon    | 90000.00 |  
| 3  | Sarthak | 20  | Niphad        | 400000.00 |  
| 4  | Shreyas | 21  | Sangamner     | 500000.00 |  
| 5  | Avej    | 23  | Loni          | 600000.00 |  
| 1  | Prajwal | 21  | Chandanapuri  | 1000000.00 |  
+---+-----+-----+-----+-----+  
5 rows in set (0.00 sec)
```



```
-- Descending order
SELECT * FROM CUSTOMERS
ORDER BY SALARY DESC;
```

-> ORDER BY SALARY DESC;

ID	NAME	AGE	ADDRESS	SALARY
1	Prajwal	21	Chandanapuri	1000000.00
5	Avej	23	Loni	600000.00
4	Shreyas	21	Sangamner	500000.00
3	Sarthak	20	Niphad	400000.00
2	Prafull	22	Chalisgaon	90000.00

5 rows in set (0.00 sec)

I) SQL ARITHMETIC OPERATORS

```
SELECT NAME, SALARY,
SALARY + 1000 AS BONUS,
SALARY - 500 AS DEDUCTION,
SALARY * 2 AS DOUBLE_SALARY,
SALARY / 2 AS HALF_SALARY,
SALARY % 2 AS REMAINDER
FROM CUSTOMERS;
```

NAME	SALARY	BONUS	DEDUCTION	DOUBLE_SALARY	HALF_SALARY	REMAINDER
Prajwal	1000000.00	1001000.00	999500.00	2000000.00	500000.000000	0.00
Prafull	90000.00	91000.00	89500.00	180000.00	45000.000000	0.00
Sarthak	400000.00	401000.00	399500.00	800000.00	200000.000000	0.00
Shreyas	500000.00	501000.00	499500.00	1000000.00	250000.000000	0.00
Avej	600000.00	601000.00	599500.00	1200000.00	300000.000000	0.00

5 rows in set (0.01 sec)